

Cours GNU/Linux

Introduction à GNU/Linux

Histoire et philosophie d'UNIX · ligne de commande · fichiers · droits · filtres · scripts shell

Sommaire

1. Mise en route
2. GNU/Linux
3. Histoire d'UNIX
4. Philosophie UNIX
5. Manipuler sous Linux
6. Manipuler des fichiers
7. Gestion des droits
8. Caractères spéciaux et filtres
9. Automatiser des tâches

Le cours complet : [README.md](#)

1. Mise en route

Accéder à un système Linux

- **Linux « live »** : démarrage depuis une clé USB **vierge** de 8 Go, préparée avec **Rufus** ou **balenaEtcher** (attention au *Secure Boot* ; système lent)
- **Machine virtuelle** : **Hyper-V** (Windows Pro, hyperviseur de type 1) ou **VirtualBox** (type 2)
- **Serveur VPS** : connexion à distance avec `ssh`
- **WSL** (*Windows Subsystem for Linux*) : un shell Linux directement sous Windows

Travaux pratiques

Ressource	Contenu
Commandes de base	Mémento des commandes de base
TP « Commandes de base Linux »	Parcours guidé, vérifié automatiquement
TP1 - Les fichiers textes	Manipuler des fichiers « texte »
TP2 - Les fichiers exécutables	<code>vim</code> , scripts et programmes
TP3 - Ligne de commande	Arborescence, informations système
TP4 - Gestion des droits	Droits sur les fichiers et répertoires
TP5 - Commandes	Utilisation avancée des commandes
Serveur LAMP	Linux, Apache, MySQL, PHP

Première séance : commandes de base, TP1, puis TP2 (en partie).

2. GNU/Linux

Linux et GNU/Linux

- **Linux** : un système d'exploitation libre construit autour du **noyau Linux**, implémentation libre d'UNIX conforme **POSIX**
- **GNU/Linux** : le noyau Linux associé aux outils du projet **GNU** (*GNU's Not UNIX*) : shell, compilateurs, bibliothèque C, commandes...
- Noyau écrit en 1991 par **Linus Torvalds**, toujours coordinateur du projet
- Diffusé sous forme de **distributions** : Debian, Ubuntu, Linux Mint, Red Hat, Fedora, Arch Linux...

Android est basé sur Linux, mais pas sur GNU.

Un logiciel libre

Les quatre libertés garanties par la licence **GNU GPL** :

1. **utiliser** le logiciel sans restriction
2. **étudier** le logiciel
3. **modifier** le logiciel pour l'adapter à ses besoins
4. **redistribuer** le logiciel sous certaines conditions précises

Un logiciel libre n'est pas nécessairement gratuit, et un logiciel gratuit n'est pas forcément libre.

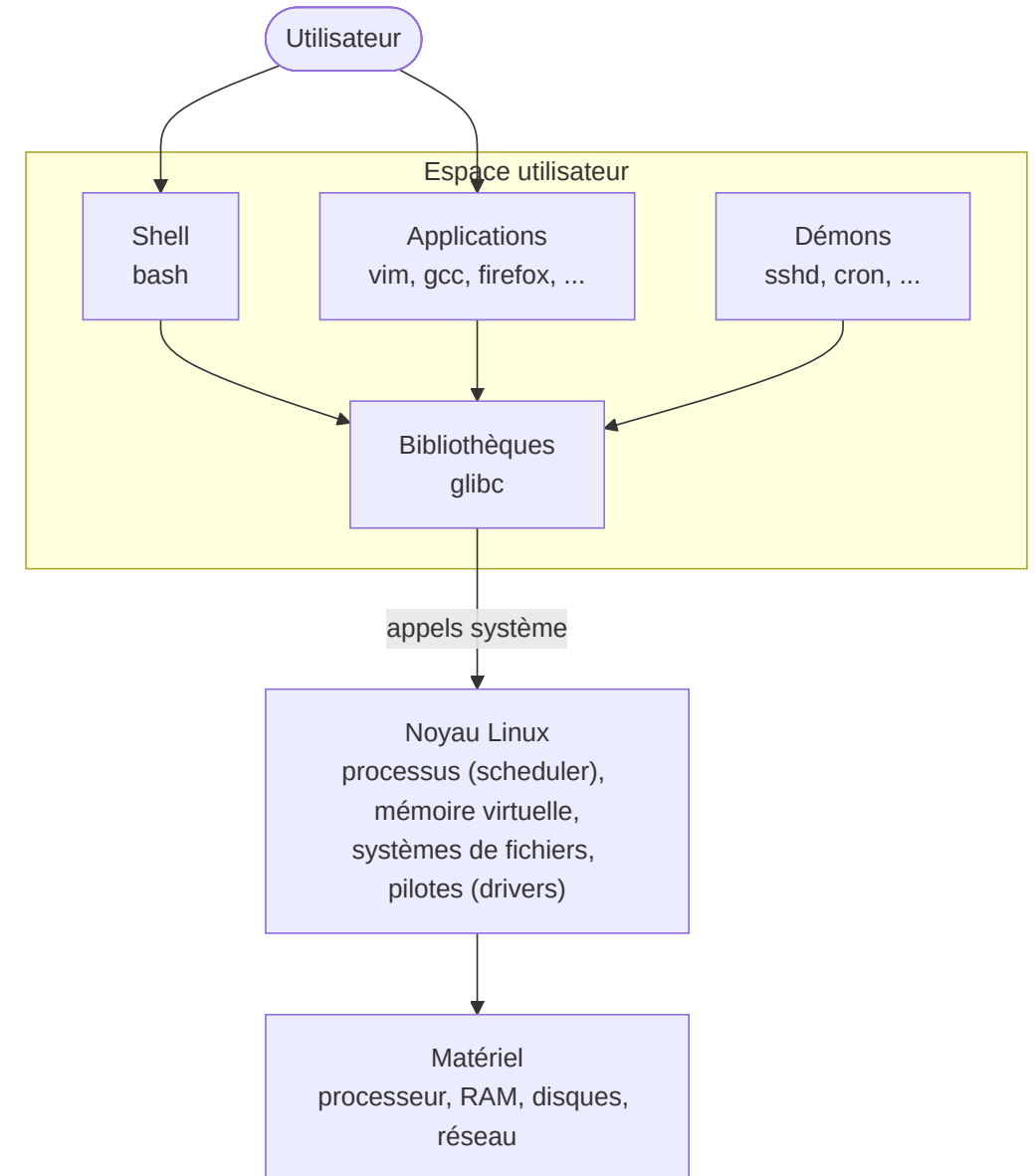
Le rôle d'un système d'exploitation

- exploite et coordonne les **périphériques** matériels
- offre aux applications des **interfaces de programmation** standardisées
- partage le **processeur** entre les processus (**multitâche**)
- gère la **mémoire**
- organise les disques en **fichiers** et **répertoires**
- fournit les **interfaces homme-machine**
- assure la **fiabilité** et la **sécurité**

UNIX est **multitâche** et **multi-utilisateur**.

Les composants de GNU/Linux

- **Noyau** : pilotes, mémoire, processus, ordonnanceur (*scheduler*)
- **Shell** : interface entre l'utilisateur et le noyau (`sh` , `csh` , `bash`)
- **Système de fichiers** : une arborescence unique à partir de `/`
- **Mémoire virtuelle** : RAM + zone d'échange (**swap**), voir `free -h`
- **Démons** : services en tâche de fond (`sshd` , `cron` , `systemd`)



3. Histoire d'UNIX

Aux origines (1964-1969)

- **Multics** (MIT, General Electric, Bell Labs) : un système à temps partagé trop complexe, abandonné par les Bell Labs en 1969
- **1969** : **Ken Thompson** et **Dennis Ritchie** écrivent un système beaucoup plus simple sur un **PDP-7** inutilisé
- Brian Kernighan le baptise *Unics*, par jeu de mots avec Multics : **UNIX** est né

UNIX et le langage C (1970-1979)

- 1970 : portage sur **PDP-11**
- **1973** : le noyau est réécrit en **C** (créé par Dennis Ritchie) : UNIX devient **portable**
- 1973 : Doug McIlroy introduit les **tubes** (*pipes*)
- AT&T n'a pas le droit de vendre de logiciels : UNIX est distribué aux universités **avec son code source**
- **1979** : la **Version 7** (Bourne shell `sh`, `awk`), ancêtre commun de tous les UNIX

BSD, l'UNIX de Berkeley

- **1977** : l'université de Berkeley distribue **BSD** (*Berkeley Software Distribution*)
- **Bill Joy** écrit `vi` et le C shell `cs``h` ; BSD fonctionne sur les **VAX** de DEC
- **1983** : 4.2BSD intègre la **pile TCP/IP** et les *sockets* : Internet se diffuse
- Procès d'AT&T (1992-1994), puis **4.4BSD-Lite**, libéré du code AT&T
- Descendants : **FreeBSD**, **NetBSD**, **OpenBSD**... et **macOS**

Les UNIX commerciaux et POSIX

- **1983** : AT&T commercialise **System V**
- Chaque constructeur a son UNIX : **Solaris** (Sun), **HP-UX** (HP), **AIX** (IBM), **IRIX** (SGI)...
- Des systèmes incompatibles : la « **guerre des UNIX** »
- **1988** : la norme **POSIX** (IEEE) définit l'interface commune d'un système de type UNIX
- « UNIX » est une marque de l'**Open Group** (*Single UNIX Specification*)

Le projet GNU

- **1983** : **Richard Stallman** annonce le projet **GNU** (*GNU's Not UNIX*) : un UNIX entièrement libre
- 1985 : *Free Software Foundation* ; **1989** : licence **GNU GPL**
- GNU écrit les outils : GCC, GDB, Emacs, `bash` , `ls` , `cp` , `grep` , la bibliothèque C...
- Il manque le **noyau** : GNU Hurd n'aboutit pas

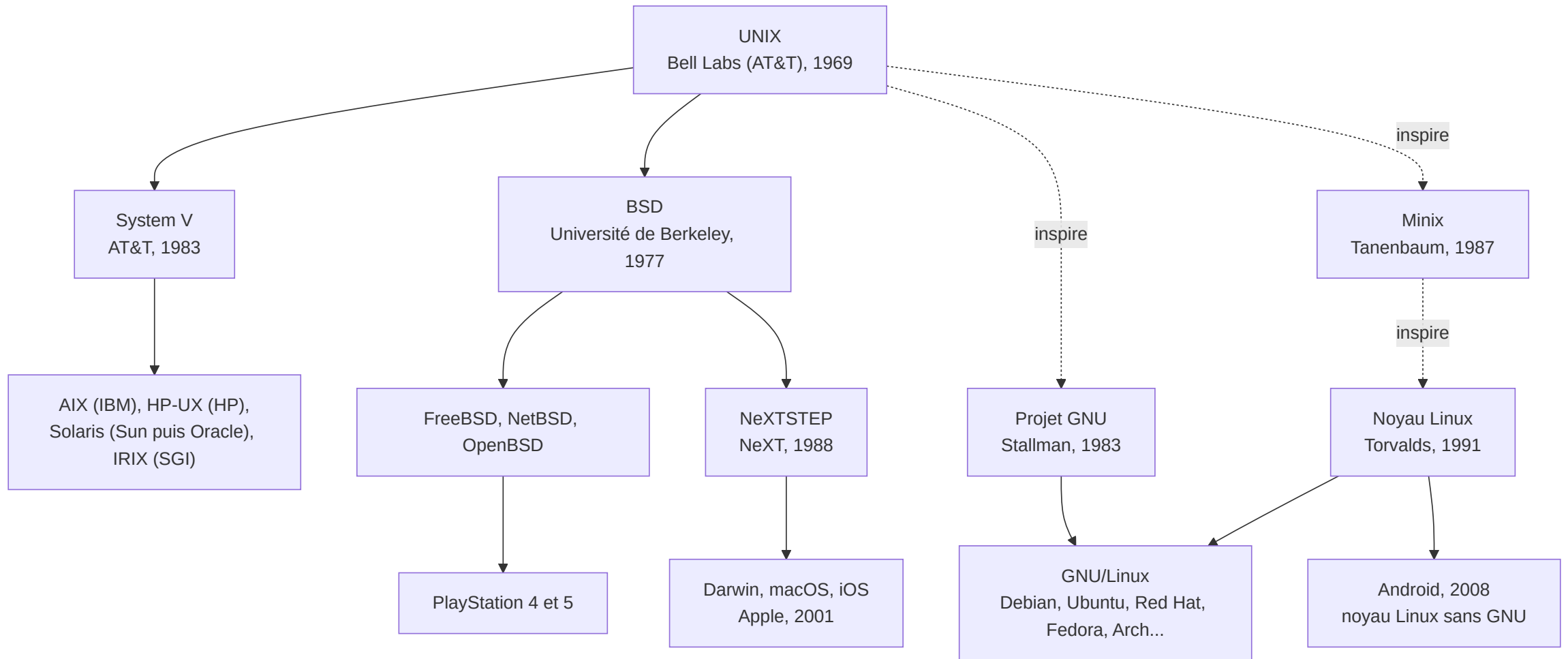
Linux (1991)

- 1987 : **Minix**, un petit UNIX d'enseignement (Andrew Tanenbaum)
- **25 août 1991** : **Linus Torvalds**, étudiant à Helsinki, annonce son noyau :

« Je fais un système d'exploitation (gratuit) (juste un passe-temps, ce ne sera pas gros et professionnel comme GNU) pour les clones AT 386(486). »

- 1992 : Linux passe sous **GPL** : noyau Linux + outils GNU = **GNU/Linux**
- Distributions : Slackware, Debian (1993), Red Hat (1994), Ubuntu (2004)...

La famille UNIX



Trois branches

Branche	Origine	Systèmes actuels
System V	AT&T	AIX, HP-UX, Solaris
BSD	Université de Berkeley	FreeBSD, NetBSD, OpenBSD, macOS, iOS
Clones (<i>Unix-like</i>)	réécrits sans le code d'AT&T	Minix, GNU/Linux, Android

- macOS est officiellement **certifié UNIX**
- Linux n'est pas certifié, mais largement compatible **POSIX**

Arbre généalogique complet : www.levenez.com/unix

Linux aujourd'hui

- la majorité des **serveurs** du web et du **cloud**
- les **500 superordinateurs** les plus puissants du monde (depuis 2017)
- **l'embarqué** : box, télévisions, voitures, objets connectés
- les **smartphones**, avec **Android** (noyau Linux, sans GNU)

Les grandes figures

Personne	Contributions
Ken Thompson	UNIX, langage B, Plan 9, UTF-8, langage Go
Dennis Ritchie	UNIX, langage C, Plan 9
Brian Kernighan	le nom « UNIX », <i>The C Programming Language</i> , <code>awk</code>
Doug McIlroy	les tubes (<i>pipes</i>), la philosophie UNIX
Bill Joy	BSD, <code>vi</code> , <code>csch</code> , TCP/IP dans BSD, Sun Microsystems
Richard Stallman	projet GNU, licence GPL, FSF, GNU Emacs, GCC
Andrew Tanenbaum	Minix
Steve Jobs	Apple, NeXT, macOS
Linus Torvalds	noyau Linux, Git

Prix Turing 1983 : Ken Thompson et Dennis Ritchie, pour UNIX.

Chronologie (1/2)

Année	Événement
1969	Premier UNIX sur PDP-7, aux Bell Labs d'AT&T
1973	UNIX réécrit en C ; les tubes
1977	Début de BSD
1979	UNIX Version 7
1983	System V ; TCP/IP dans BSD ; projet GNU
1985	Free Software Foundation ; Steve Jobs fonde NeXT
1987	Minix
1988	Norme POSIX
1989	Licence GPL ; shell <code>bash</code>
1990	Tim Berners-Lee crée le Web au CERN sur un NeXT
1991	Linus Torvalds annonce Linux

Chronologie (2/2)

Année	Événement
1992	Linux sous licence GPL ; encodage UTF-8
1993	Slackware, Debian, FreeBSD, NetBSD
1994	Linux 1.0 ; 4.4BSD-Lite
1995	OpenBSD
1998	Apparition du terme « open source »
2001	Mac OS X, basé sur Darwin (BSD)
2004	Ubuntu
2005	Linus Torvalds crée Git
2008	Premier smartphone Android
2017	Linux équipe les 500 superordinateurs les plus puissants
2019	UNIX fête ses 50 ans

Culture du logiciel libre

- **Licence GPL** (1989) : *copyleft*, un logiciel modifié et redistribué reste sous GPL ; les licences **BSD** et **MIT** sont *permissives* (réutilisées par macOS, la PlayStation)
- **La cathédrale et le bazar** (Eric S. Raymond, 1997) : « avec suffisamment d'yeux, tous les bugs sont superficiels »
- **Homesteading the Noosphere** (Eric S. Raymond, 1998) : propriété et culture du don dans l'open source
- **Déclaration d'indépendance du cyberspace** (John Perry Barlow, 1996)

UNIX en images et en vidéos

- Vidéo : [AT&T Archives: The UNIX Operating System](#) (1982)
- Vidéo : [Where GREP Came From - Computerphile](#)
- Les machines d'UNIX : [PDP-7](#), [PDP-11](#), [VAX](#) (photo d'un [VAX 11/780](#))
- [L'Apple I](#) (1976)
- [Le NeXT de Tim Berners-Lee](#), premier serveur web
- [L'annonce de Linux](#) (1991)

4. Philosophie UNIX

« Less is more »

- Des programmes qui effectuent **une seule chose** et qui la font bien
- **Le silence est d'or** : un programme qui réussit n'affiche rien
- Des programmes qui **collaborent**
- Des programmes qui gèrent des **flux de texte**, l'interface universelle

Citations

« Il est plus facile de définir un système d'exploitation par ce qu'il fait que par ce qu'il est. » **J.L. Peterson**

« Unix est convivial. Cependant Unix ne précise pas vraiment avec qui. » **Steven King**

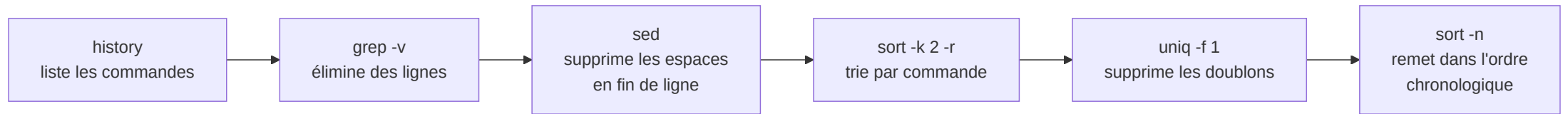
« Unix ne dit jamais 's'il vous plaît'. » **Rob Pike**

« Unix est simple. Il faut juste être un génie pour comprendre sa simplicité. » **Dennis Ritchie**

« Unix n'a pas été conçu pour empêcher ses utilisateurs de commettre des actes stupides, car cela les empêcherait aussi des actes ingénieux. » **Doug Gwyn**

Qu'est-ce qu'un UNIX ?

```
$ history | grep -v " h" | sed 's/[ \t]*$//' | sort -k 2 -r | uniq -f 1 | sort -n
```



Chaque programme fait une seule chose ; le tube `|` les fait collaborer. Résultat : l'historique des commandes, sans les doublons.

5. Manipuler sous Linux

Interfaces et conventions

- **GUI** (*Graphical User Interface*) : fenêtres, souris, icônes, menus
- **CLI** (*Command Line Interface*) : puissante, rapide, peu gourmande ; beaucoup de serveurs ne s'administrent qu'en ligne de commande
- L'interpréteur de commandes est le **shell**

Dans les exemples, l'invite (*prompt*) indique les droits nécessaires :

- `$ commande` : utilisateur ordinaire
- `# commande` : super-utilisateur *root*

Ne jamais taper l'invite `$` ou `#` !

Conseils

- **Tab** : complétion des commandes, des options, des noms de fichiers et des variables
- ↑ et ↓ : parcourir l'historique des commandes
- **Ctrl + R** : rechercher dans l'historique
- Travailler en **plein écran**, sans souris
- Utiliser **plusieurs terminaux** : un pour les commandes, un pour le manuel
 - onglets : `Ctrl + Page↑` / `Ctrl + Page↓` ; fenêtres : `Alt + Tab`

Structure d'une commande

```
$ commande [options] <paramètres>
```

- des mots séparés par des espaces
- `[ ]` : facultatif (on ne tape pas les crochets)
- options courtes `-l` ou longues `--all` (standard GNU)
- l'ordre des options a rarement de l'importance

```
$ ls --all -l --si  
$ ls -l --si --all  
$ ls -l $HOME/tmp
```

Types de commandes

- **internes** au shell : `cd`, `echo`, `history`, `test`
- **externes** (programmes) : `ls`, `mkdir` ... dans `/bin`, `/usr/bin`, `/sbin`
- **alias** : `ll`
- `$PATH` : la liste des répertoires où le shell cherche les commandes

```
$ type echo
echo est une primitive du shell
$ type strings
strings est /usr/bin/strings
$ type ll
ll est un alias vers « ls -halF »
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Obtenir de l'aide

- `man ls` : la page de manuel (`/motif` pour chercher, `n` / `N` , `q` pour quitter)
- sections : 1 commandes, 2 appels système, 3 fonctions C, 5 formats de fichiers, 8 administration
- `man 1 mkdir` et `man 2 mkdir` : deux pages différentes
- `apropos mot-clé` : chercher dans tout le manuel ; `whatis ls` : description courte
- `help cd` (commandes internes), `ls --help` , `info ls`

RTFM : *Read The Fine Manual*

Le shell Bash et les sessions

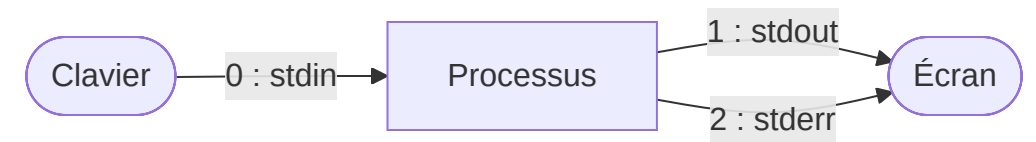
- **Bash** (*Bourne-again shell*) : le shell du projet GNU, par défaut sur la plupart des distributions ; autres shells : `sh` , `ksh` , `csch` , `zsh` ...
- ouvrir une session : `login` , `su` (changer d'utilisateur), `ssh` (à distance, chiffré)
- l'invite est définie par la variable `PS1`
- fermer une session : `exit` , `logout` ou **Ctrl + D**

Les flux standard

Tout processus démarre avec trois flux :

n°	Flux	Par défaut
0	<i>stdin</i>	le clavier
1	<i>stdout</i>	l'écran
2	<i>stderr</i>	l'écran

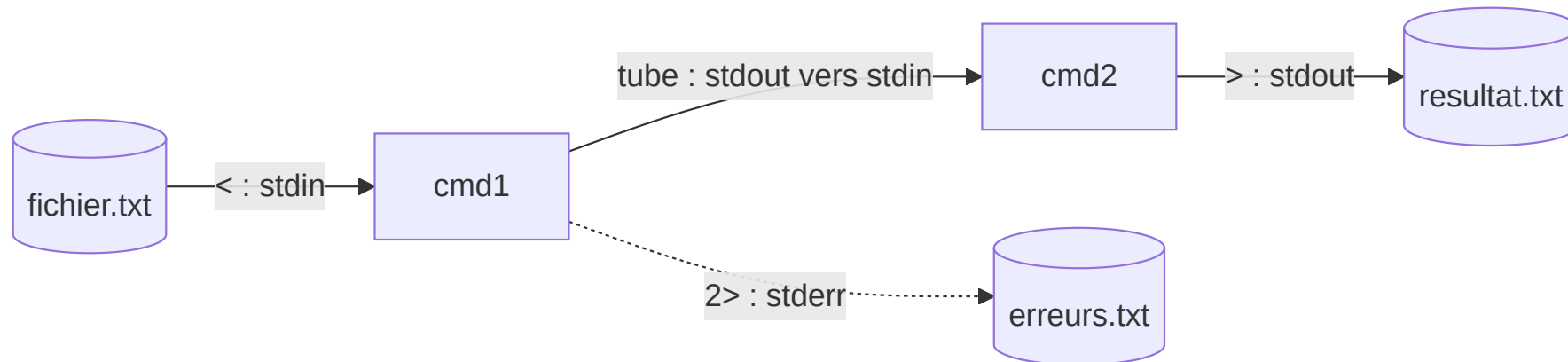
Un **processus** est un programme en cours d'exécution, identifié par un PID.



Redirections et tubes

```
$ ls > liste.txt  
$ date >> journal.txt  
$ wc -l < liste.txt  
$ find / -name passwd 2> /dev/null  
$ cat bonjour.txt | wc -c
```

stdout vers un fichier (écrasé)
stdout ajouté à la fin du fichier
stdin depuis un fichier
stderr supprimé
tube : stdout de cat vers stdin de wc

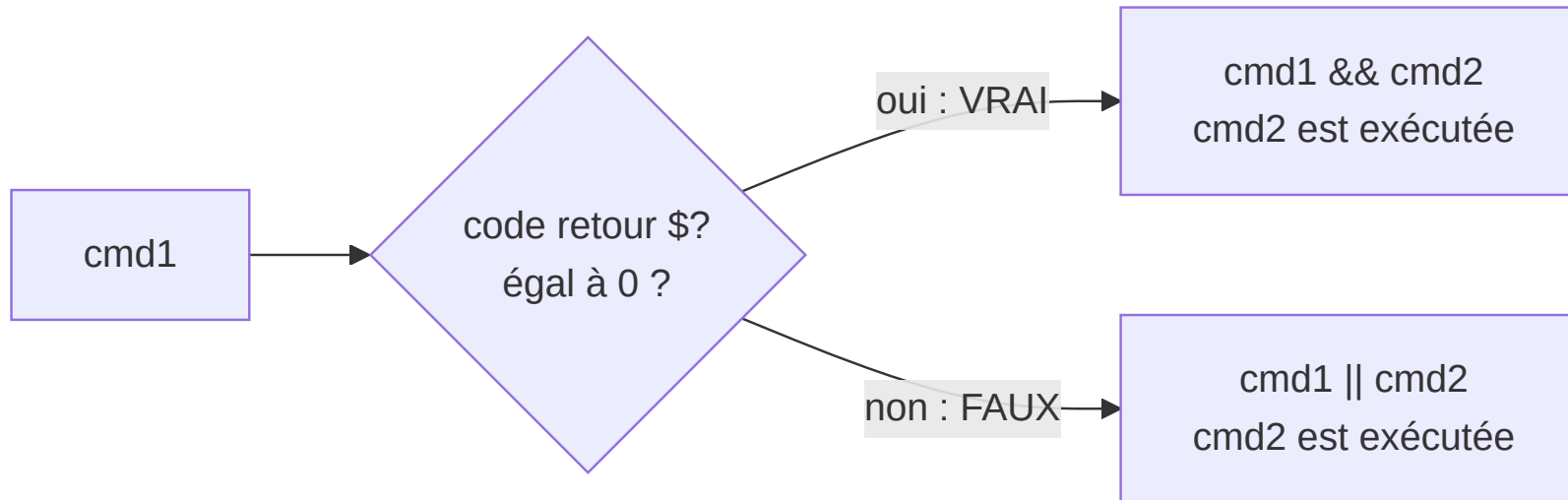


L'historique des commandes

```
$ history                # tout l'historique
$ history | grep ls      # rechercher (voir aussi Ctrl + R)
$ !!                     # relance la dernière commande
$ !ls                    # relance la dernière commande commençant par ls
$ !100                   # relance la commande n°100
$ echo !$                # !$ : dernier argument de la commande précédente
$ history -c             # efface l'historique
```

Groupement de commandes

```
cmd1 ; cmd2      # cmd1 puis cmd2
cmd1 | cmd2      # tube entre cmd1 et cmd2
cmd1 && cmd2      # cmd2 seulement si cmd1 réussit
cmd1 || cmd2     # cmd2 seulement si cmd1 échoue
```



```
$ rm fff || echo "Houston, on a un problème !"
```

Le code de retour `$?`

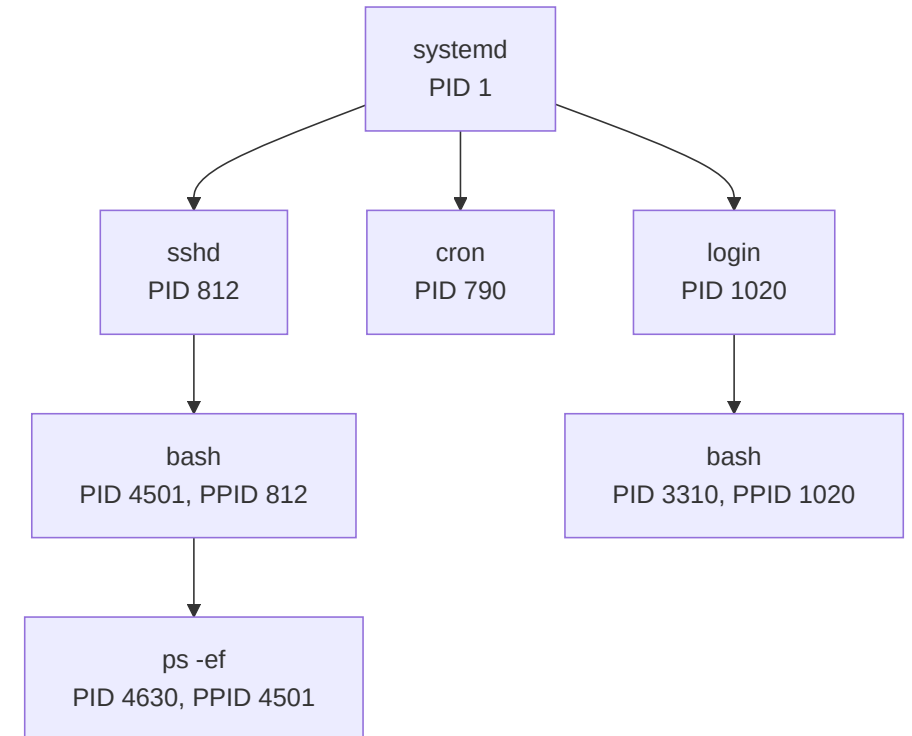
Chaque commande renvoie un **code de retour** : `0` = succès (VRAI), autre valeur = erreur (FAUX)

```
$ ls ; echo $?
0
$ ls zzz ; echo $?
2
$ touch test.log
$ test -s test.log || echo "le fichier est vide"
le fichier est vide
$ test -e test.log && echo "le fichier existe"
le fichier existe
```

Les processus

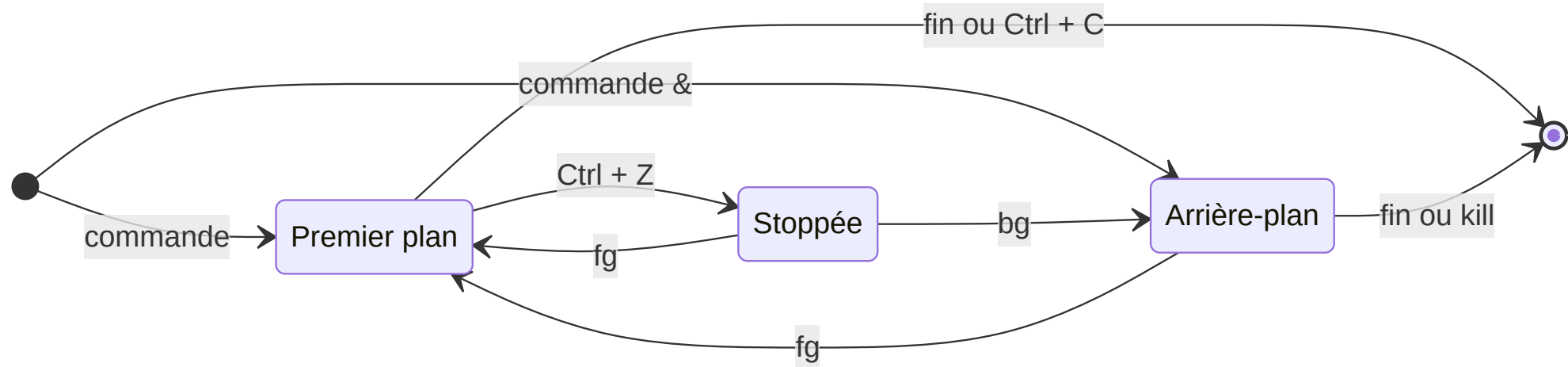
- un **processus** : un programme en cours d'exécution
- identifié par un **PID**, il connaît son parent (**PPID**)
- ancêtre de tous : le **PID 1** (`systemd` , autrefois `init`)
- l'**ordonnanceur** du noyau partage le processeur

```
$ ps -ef           # tous les processus
$ pstree           # l'arbre des processus
$ top              # en temps réel
$ pidof bash       # PID d'un programme
```



Gérer les tâches

```
$ sleep 300 &      # lance en arrière-plan  
[1] 12345  
$ jobs            # tâches du shell  
$ kill %1         # signal TERM (kill -9 : KILL)
```



`nohup` : détacher du terminal · `at` , `batch` : exécution différée · `cron` : planification

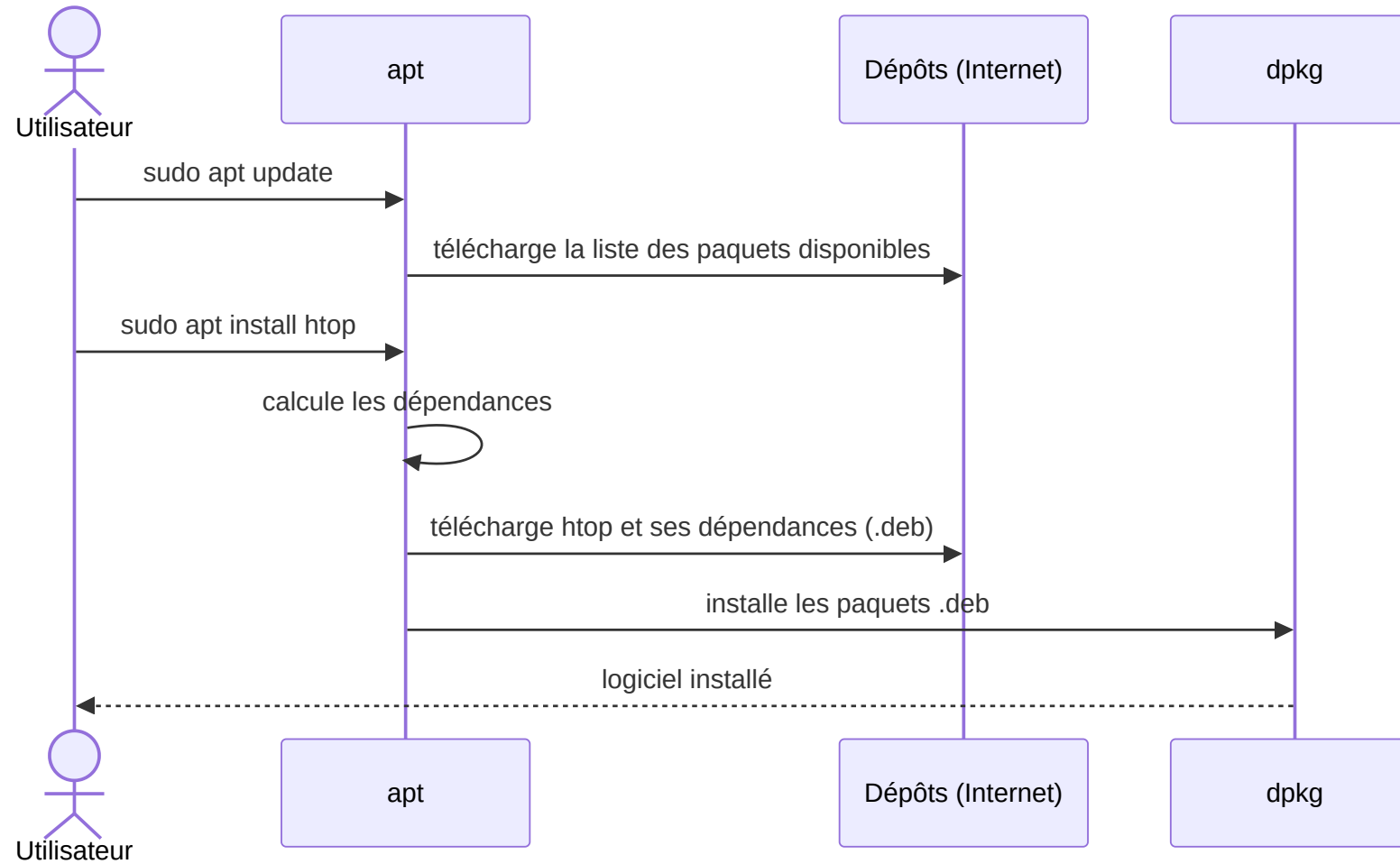
Installer des logiciels : les paquets

- des **paquets** (`.deb` sur Debian et Ubuntu) téléchargés depuis des **dépôts**
- `dpkg` (bas niveau), **APT** (`apt`, `apt-get` : gère les dépendances), `synaptic` (graphique)

```
$ sudo apt update           # liste des paquets disponibles
$ sudo apt upgrade         # met à jour le système
$ apt search htop          # recherche un paquet
$ sudo apt install htop    # installe un paquet et ses dépendances
$ sudo apt remove htop     # supprime un paquet
$ dpkg -L htop             # fichiers installés par le paquet
```

Autres familles : RPM et `dnf` (Red Hat, Fedora), `pacman` (Arch Linux)

APT en action



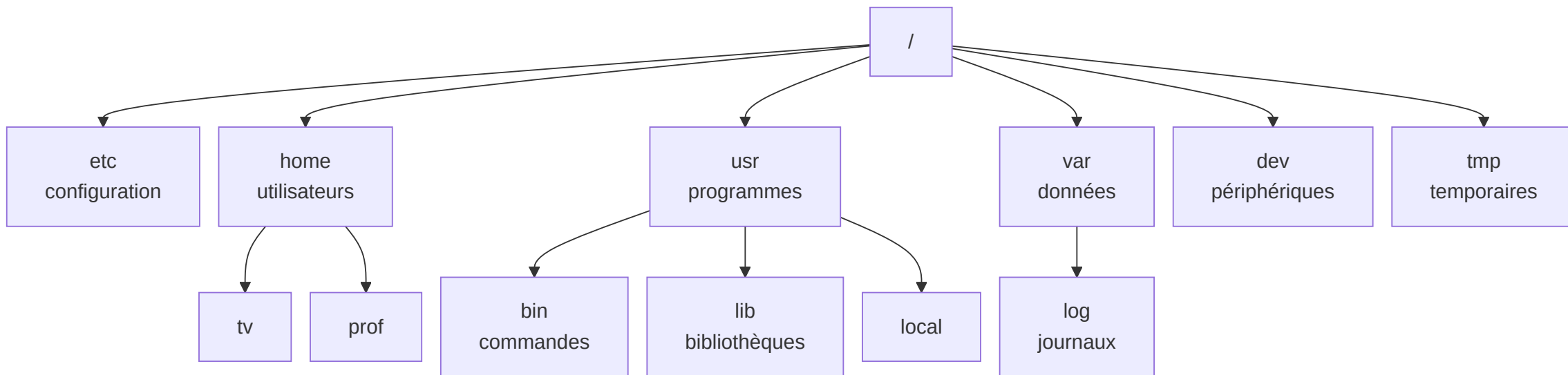
6. Manipuler des fichiers

Le système de fichiers

- organise les données en **fichiers** et **répertoires** sur un support (disque, SSD, clé USB)
- on **partitionne** le disque (`fdisk`), puis on **formate** chaque partition
- types : **ext4** (Linux), Btrfs, XFS, FAT, exFAT, NTFS, APFS, ISO 9660...
- sous UNIX : une **arborescence unique** à partir de la racine `/`

```
$ df -Th          # systèmes de fichiers montés et leur type
```

L'arborescence



Autres répertoires : `/bin` et `/sbin` (commandes), `/boot` (noyau, démarrage), `/proc` et `/sys` (noyau et processus), `/root` (répertoire de root)

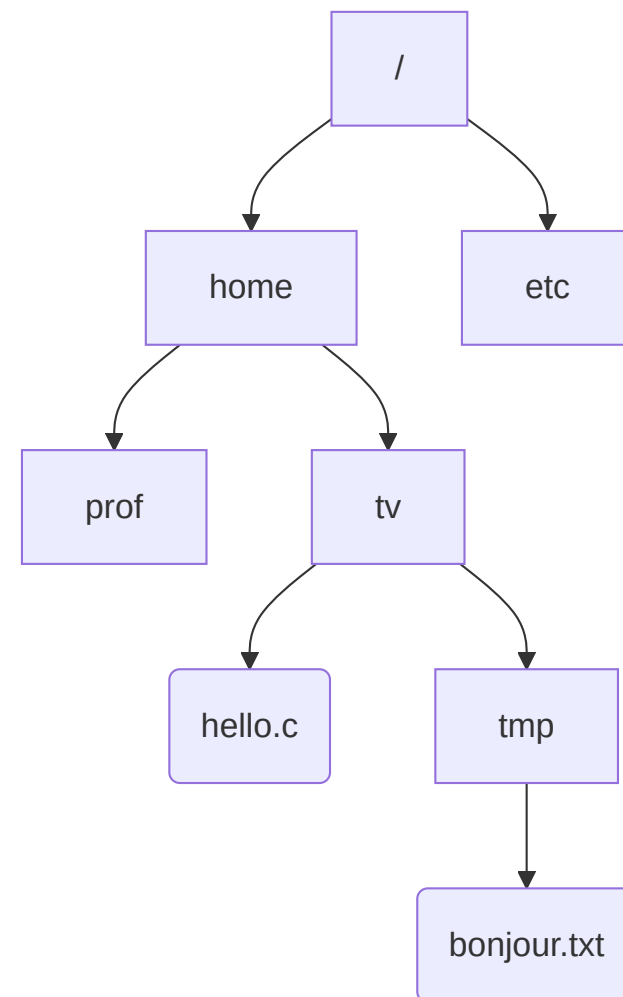
Norme **FHS** (*Filesystem Hierarchy Standard*)

Chemins absolus et relatifs

- **absolu** : depuis la racine, commence toujours par `/`
- **relatif** : depuis le répertoire courant ; `.` courant, `..` parent, `~` personnel

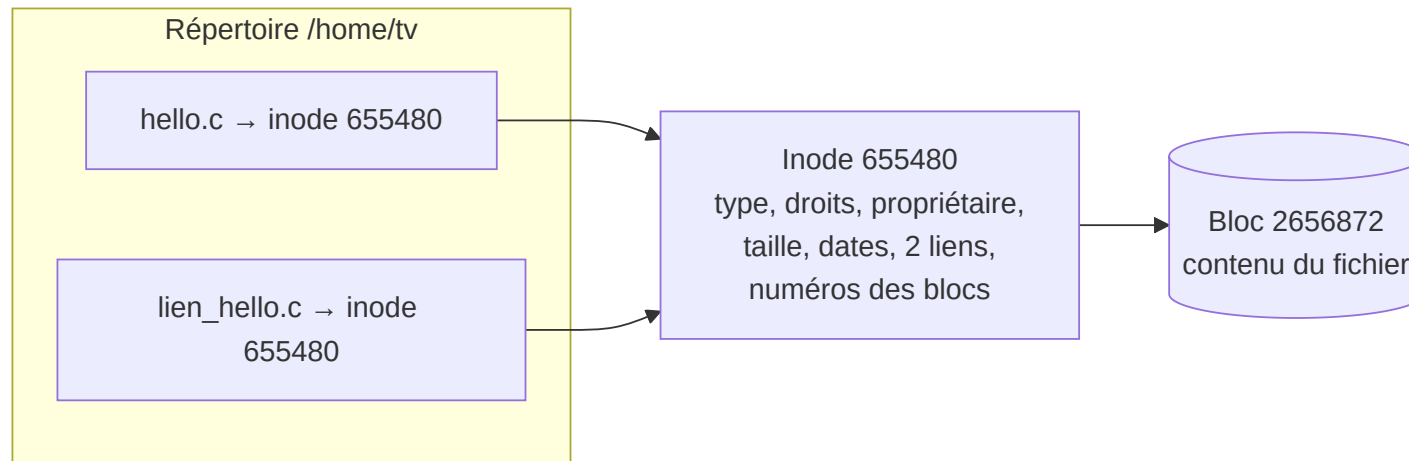
Répertoire courant	hello.c	bonjour.txt
(n'importe lequel)	<code>/home/tv/hello.c</code>	<code>/home/tv/tmp/bonjour.txt</code>
<code>/home/tv</code>	<code>./hello.c</code>	<code>./tmp/bonjour.txt</code>
<code>/home/prof</code>	<code>../tv/hello.c</code>	<code>../tv/tmp/bonjour.txt</code>

```
$ pwd          # affiche le répertoire courant
$ cd ..        # remonte au répertoire parent
$ cd           # retourne dans le répertoire personnel
```



Fichiers et inodes

- un fichier : une suite d'**octets** portant un nom
- son **inode** contient ses **métadonnées** : type, droits, propriétaire, taille, dates, nombre de liens, numéros des blocs
- le **nom** est dans le **répertoire**, pas dans l'inode



```
$ ls -li fichier ; stat fichier
```

Fichiers texte et binaires

- **texte** : des caractères encodés (ASCII, UTF-8) : code source, scripts, configuration... On les modifie avec un **éditeur de texte**
- **binaire** : tout le reste : exécutables, images, sons, bases de données...
- **compressé** (`gzip` , `xz` , `zip`) et **archive** (`tar` , `.tar.gz`)
- fins de ligne : **LF** (UNIX), **CRLF** (Windows, protocoles Internet), **CR** (anciens Mac)

```
$ od -c bonjour.txt           # voir les octets, dont le \n final
$ hexdump -C bonjour.txt
```

L'encodage des caractères

- **ASCII** : 7 bits, 128 caractères, sans accents
- **ISO 8859-1** (*latin1*), **ISO 8859-15** (*latin9*, avec « œ » et « € »)
- **Unicode** : plus de 150 000 caractères, encodés en **UTF-8** (UNIX, Internet) ou en UTF-16 (Java, Windows) ; UTF-8 est compatible avec ASCII

```
$ file bonjour.txt
bonjour.txt: UTF-8 Unicode text
$ iconv -f UTF-8 -t ISO8859-1 bonjour.txt -o bonjour_latin1.txt
$ man ascii
```

Pas d'espaces ni de caractères accentués dans les noms de fichiers !

Créer et afficher

```
$ mkdir tmp ; cd tmp ; pwd
/home/tv/tmp
$ touch vide                                # fichier vide
$ echo "Hello world" > bonjour.txt          # fichier avec un contenu
$ echo "by $USER" >> bonjour.txt            # ajout à la fin
$ cat bonjour.txt                          # afficher (cat -n : numéroté)
$ less bonjour.txt                         # page par page (q pour quitter)
$ head -1 bonjour.txt                     # première ligne
$ tail -2 bonjour.txt                     # deux dernières lignes
$ wc -l bonjour.txt                       # nombre de lignes
```

Copier, déplacer, supprimer, rechercher

```
$ cp /etc/passwd .           # copier
$ cp -r temp temp1          # copier un répertoire
$ mv utilisateurs liste.txt  # renommer ou déplacer
$ rm liste.txt              # supprimer
$ rm -r temp                # supprimer un répertoire
$ find $HOME -name "*.txt"   # rechercher
$ find $HOME -name "*.txt" -exec ls -l {} \;
```

Pas de corbeille en ligne de commande : `rm` est **définitif**.

Le motif de `find` doit être **entre guillemets**.

L'éditeur vim

- mode **normal** au démarrage ; **i** : mode **insertion** ; **Esc** : retour au mode normal ; **:** : mode **commande**

Commande	Action
:w / :q! / :wq	sauvegarder / quitter sans sauvegarder / les deux
dd / yy / p	couper / copier / coller une ligne
u	annuler
/mot puis n / N	rechercher
:set nu	afficher les numéros de ligne

vimtutor : un tutoriel interactif de 30 minutes

7. Gestion des droits

Utilisateurs et groupes

- un utilisateur : un nom, un **UID**, un groupe principal (**GID**), des groupes secondaires
- **root** : UID 0, aucune restriction
- `/etc/passwd` (comptes), `/etc/shadow` (mots de passe hachés), `/etc/group` (groupes)

```
$ id
uid=1000(tv) gid=1000(tv) groupes=1000(tv),4(adm),27(sudo)
$ grep tv /etc/passwd
tv:x:1000:1000:,,,:/home/tv:/bin/bash
```

Commandes : `id`, `groups`, `whoami`, `who`, `w`, `last`

Lire les permissions

```
$ ls -l script.sh
-rwxr-x--- 1 tv promo00 38 sept. 26 10:12 script.sh
```

Type	Utilisateur (u)	Groupe (g)	Autres (o)	Propriétaire	Groupe propriétaire
-	rwx	r-x	---	tv	promo00

Types : - fichier · d répertoire · l lien symbolique · c / b périphérique · s socket · p tube nommé

Chaque bloc de trois caractères donne les droits r (lecture), w (écriture) et x (exécution) ; un - indique un droit absent.

Les permissions de base

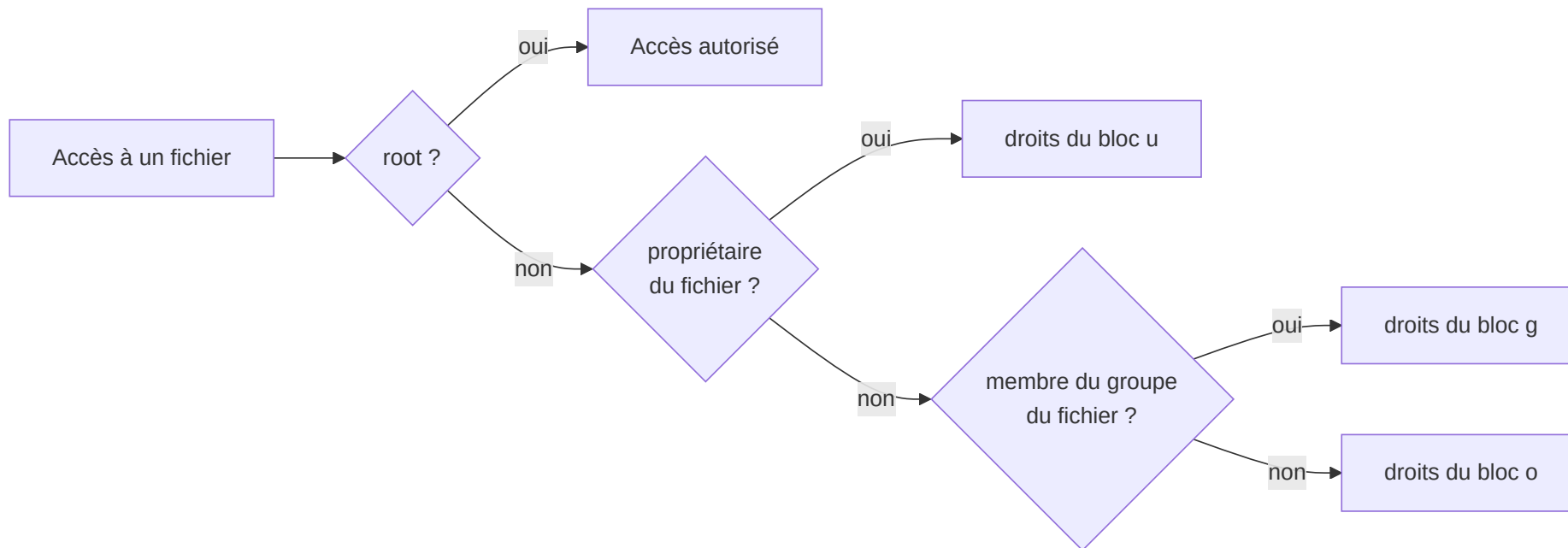
Droit	Sur un fichier	Sur un répertoire
r	lire le contenu	lister le contenu (<code>ls</code>)
w	modifier le contenu	créer, supprimer, renommer des fichiers
x	exécuter (programme, script)	traverser (<code>cd</code> , accès aux fichiers)

Attribuées à : `u` l'utilisateur · `g` le groupe · `o` les autres · `a` tout le monde

■ Pour supprimer un fichier, il faut le droit `w` sur le **répertoire**, pas sur le fichier.

Qui a quels droits ?

- les droits sont vérifiés dans l'ordre **u**, **g**, **o** : dès qu'une catégorie correspond, **seuls ses droits** s'appliquent
- avec `----rwx---` (`chmod 070`), le propriétaire n'a **aucun** accès, même s'il fait partie du groupe



Modifier les permissions : chmod

```
$ chmod g+rx fichier      # ajoute r et x au groupe
$ chmod a-x fichier      # retire x à tout le monde
$ chmod u=rw fichier     # fixe les droits de l'utilisateur
$ chmod 750 fichier      # rwxr-x---
$ chmod -R g+w projet    # récursivement
```

Octal	0	1	2	3	4	5	6	7
Droits	---	--x	-w-	-wx	r--	r-x	rw-	rwx

$r = 4$, $w = 2$, $x = 1$: $644 = \text{rw-r--r--}$, $755 = \text{rwxr-xr-x}$

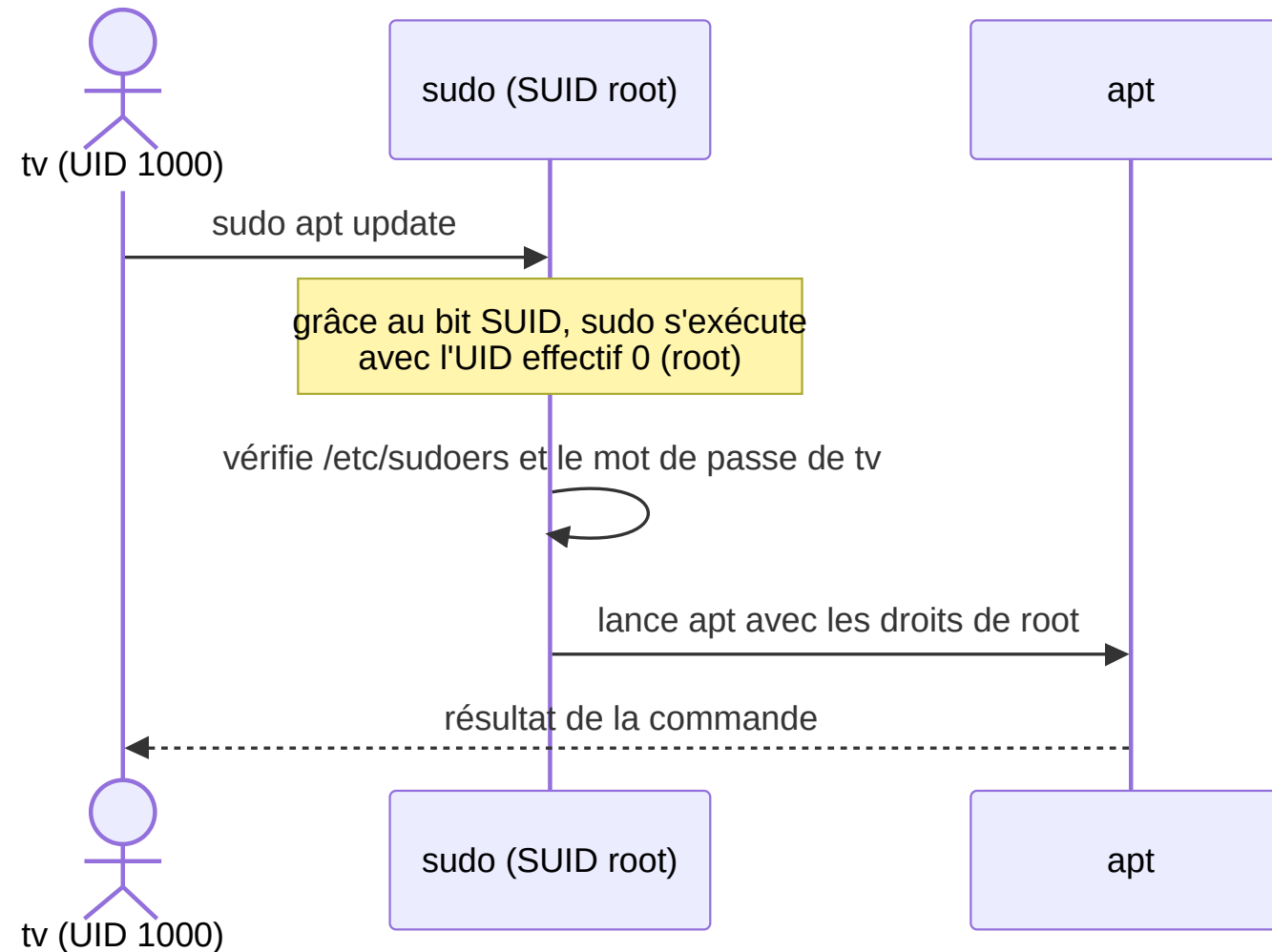
Les droits spéciaux

Droit	Sur un exécutable	Sur un répertoire	Octal
SUID <code>s</code> (bloc u)	s'exécute avec l'UID du propriétaire	aucun effet	4
SGID <code>s</code> (bloc g)	s'exécute avec le GID du groupe	les fichiers créés héritent du groupe	2
sticky bit <code>t</code> (bloc o)	ignoré par Linux	seul le propriétaire d'un fichier peut le supprimer	1

```
$ ls -l /usr/bin/sudo
-rwsr-xr-x 2 root root 70K mars 12 17:35 /usr/bin/sudo
$ ls -ld /tmp
drwxrwxrwt 20 root root 4096 sept. 26 10:12 /tmp
$ chmod 2775 projet # répertoire d'équipe : rwxrwsr-x
```

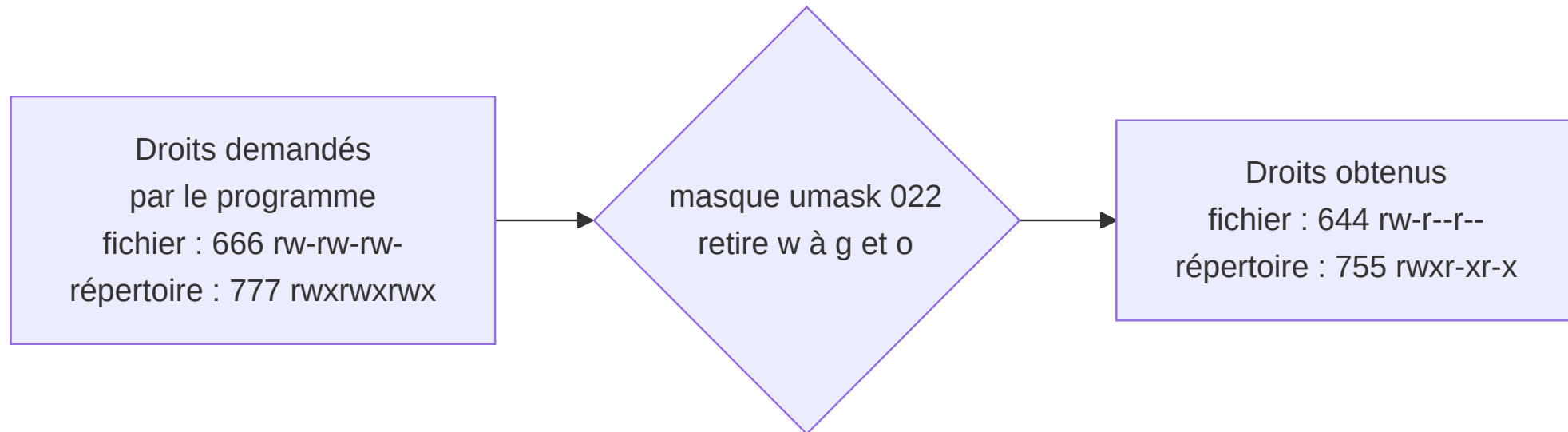
Attention : un SUID mal placé est une faille de sécurité (jamais sur `cat` !).

SUID : l'exemple de sudo



Les droits par défaut : umask

```
$ umask
0022
$ umask -S
u=rwx,g=rx,o=rx
```



Fichier : $666 \& \sim 022 = \mathbf{644}$ · répertoire : $777 \& \sim 022 = \mathbf{755}$

chown, chgrp et pour aller plus loin

```
# chown tv fichier          # propriétaire (root uniquement)
# chown tv:promo00 fichier  # propriétaire et groupe
$ chgrp promo00 fichier     # groupe (dont on est membre)
```

- **sudo** : exécuter une commande en tant que root (`/etc/sudoers`)
- **ACL** : des droits pour d'autres utilisateurs ou groupes (`getfacl` , `setfacl`)
- Vidéos : [Sticky bit, SetUID, SetGID](#) · [Special Linux Permissions](#) · [umask](#)
- [TP4 - Gestion des droits](#)

8. Caractères spéciaux et filtres

Les caractères génériques du shell

Interprétés **par le shell**, avant le lancement de la commande :

Motif	Désigne
<code>*</code>	n'importe quelle chaîne, même vide
<code>?</code>	un caractère quelconque
<code>[abc]</code> , <code>[a-c]</code>	un caractère de la liste
<code>[!abc]</code>	un caractère hors de la liste
<code>{abc,cod}</code>	l'une des chaînes

```
$ ls *.txt
```

```
$ ls a?c*
```

```
$ ls [a-c]*
```

```
$ ls *.{c,txt}
```

Les expressions régulières : opérateurs

Un **motif** (*pattern*) décrit un ensemble de chaînes : `grep` , `sed` , `awk` , `vim` ...

Opérateur	Signification	Exemple	Correspond à
<code>.</code>	un caractère quelconque	<code>a.c</code>	« abc », « a2c »
<code>[...]</code>	un caractère de la liste	<code>[aeiou]</code> , <code>[a-d]</code>	« a », « e »
<code>[^...]</code>	un caractère hors de la liste	<code>[^aeiou]</code>	« b », « c »
<code>^</code>	début de ligne	<code>^a</code>	« a » en début de ligne
<code>\$</code>	fin de ligne	<code>a\$</code>	« a » en fin de ligne
<code> </code>	alternative	<code>a b</code>	« a » ou « b »
<code>(...)</code>	groupement	<code>(ab)+</code>	« ab », « abab »
<code>\</code>	échappement	<code>\.</code>	un point

Les expressions régulières : quantificateurs

Quantificateur	Signification	Exemple	Correspond à	Pas à
<code>?</code>	zéro ou une fois	<code>toto?</code>	« tot », « toto »	« totoo »
<code>*</code>	zéro, une ou plusieurs fois	<code>toto*</code>	« tot », « toto », « totoo »	
<code>+</code>	une ou plusieurs fois	<code>toto+</code>	« toto », « totoo »	« tot »
<code>{n}</code>	exactement n fois	<code>a{3}</code>	« aaa »	« aa »
<code>{n,m}</code>	entre n et m fois	<code>a{2,4}</code>	« aa », « aaaa »	« a »
<code>{n,}</code>	au moins n fois	<code>a{3,}</code>	« aaa », « aaaa »	« aa »

Classes POSIX : `[:digit:]` `[:alpha:]` `[:alnum:]` `[:upper:]` `[:lower:]` `[:space:]`

BRE, ERE et PCRE

- **BRE** (par défaut pour `grep` et `sed`) : `\?` `\+` `\{ \}` `\( \)` `\|` doivent être échappés
- **ERE** (`grep -E`, `sed -E`) : ces caractères sont spéciaux sans échappement
- **PCRE** (`grep -P`) : la syntaxe de Perl

```
$ grep '\(84\|13\)[[:digit:]]\{3\}' liste.txt      # BRE
$ grep -E '(84|13)[[:digit:]]{3}' liste.txt      # ERE
$ sed -En '/(84|13)[[:digit:]]{3}/p' liste.txt    # ERE
Sarrians 84260
Avignon 84000
...
```

Protéger les caractères spéciaux

Protection	Effet
<code>\</code>	annule le caractère suivant
<code>'...'</code>	annule tous les caractères spéciaux
<code>"..."</code>	annule tout sauf <code>\$</code> , <code>`</code> et <code>\</code>

```
$ nom=tv
$ echo "hello $nom"      # hello tv
$ echo 'hello $nom'      # hello $nom
$ echo "hello \$nom"     # hello $nom
```

Les filtres grep, sed et awk

```
$ grep bash /etc/passwd           # lignes contenant bash
$ grep -v '^#' fichier.conf       # lignes ne commençant pas par #
$ grep -c '[:upper:]' fichier     # lignes contenant une majuscule
$ sed 's/Windows/Linux/g' texte.txt # remplacer
$ sed '/^$/d' texte.txt           # supprimer les lignes vides
$ df | sed 1d | awk '{print $1 " = " $4}' # colonnes 1 et 4
$ cut -d: -f1,7 /etc/passwd       # champs 1 et 7
```

egrep et **fgrep** sont obsolètes : on utilise **grep -E** et **grep -F**.

9. Automatiser des tâches

Pourquoi des scripts ?

- automatiser les tâches **répétitives** : sauvegarde, surveillance, installation
- simplifier l'utilisation de logiciels complexes
- mémoriser des commandes aux nombreuses options
- planifier des traitements (`at` , `cron`)

Un **script shell** est un fichier **texte**, **exécutable** (droit `x`) et **interprété** par un shell.

Premier script

```
$ vim script.sh
```

```
#!/bin/bash  
echo "je suis un script"
```

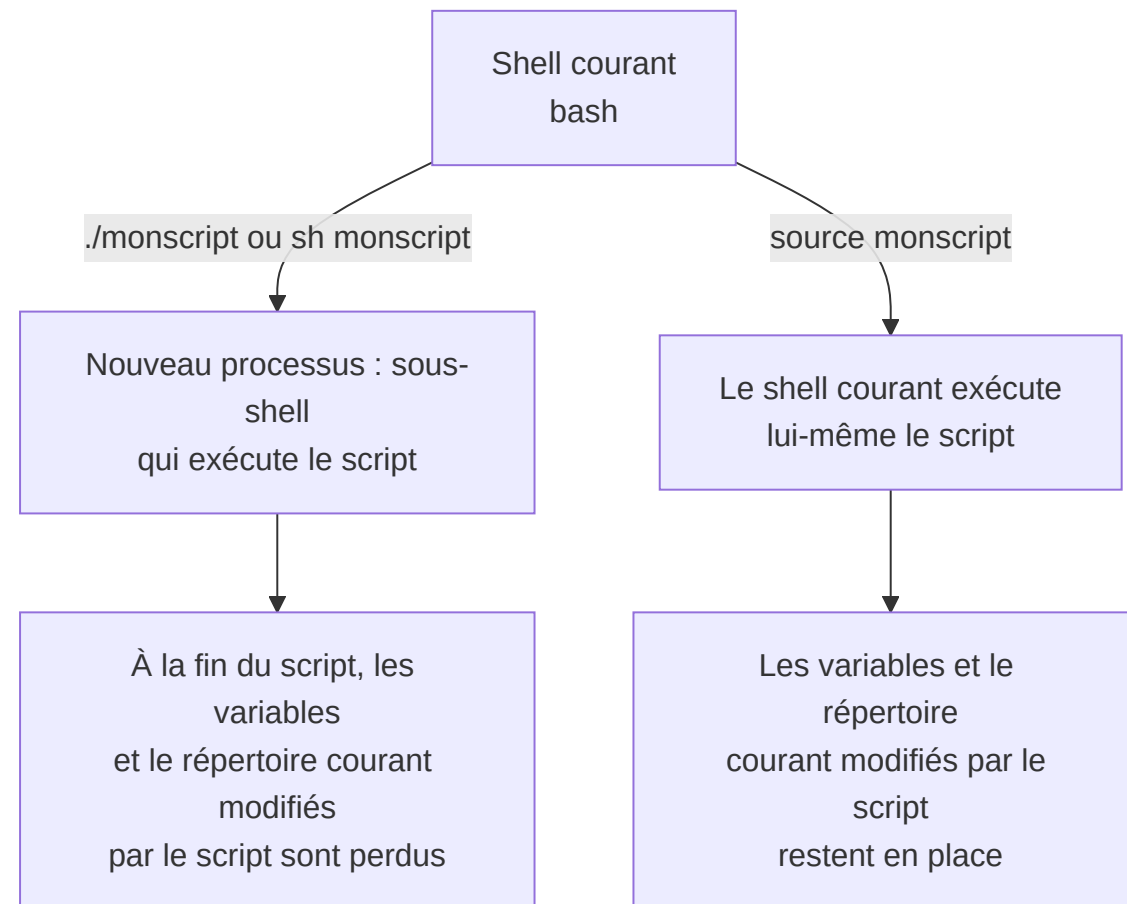
```
$ chmod +x script.sh  
$ ./script.sh  
je suis un script
```

La première ligne, le **shebang** `#!`, indique l'interpréteur à utiliser.

Exécuter un script

```
$ ./monscript      # sous-shell  
$ sh monscript     # sous-shell  
$ source monscript # shell courant
```

- `./monscript` nécessite le droit `x`
- avec `source` (ou `.`), les variables et le répertoire courant modifiés par le script restent en place



Les variables

```
$ chaine=bonjour          # pas d'espace autour de =
$ echo $chaine ${chaine}
$ nom=tv
$ echo "hello $nom"       # hello tv
$ unset chaine            # détruit la variable
$ export EDITOR=vim       # visible par les programmes lancés
$ echo $HOME $USER $PATH  # variables d'environnement
```

- une variable existe dès qu'on lui donne une valeur
- par défaut, tout est **chaîne de caractères** (typage faible)

Les substitutions

```
$ fichier=texte.htm
$ echo ${#fichier}           # longueur : 9
$ echo ${fichier%htm}html    # texte.html
$ echo ${fichier##*.}        # extension : htm
$ echo ${nom:-inconnu}       # valeur par défaut
$ NB=$(ls | wc -l)           # substitution de commande
$ somme=$((2 + 3))            # évaluation arithmétique
$ echo "scale=2; 2500/1000" | bc -l # calcul sur les réels : 2.50
```

Paramètres et variables internes

Variable	Contenu
<code>\$0</code>	nom du script
<code>\$1</code> , <code>\$2</code> , ...	paramètres
<code>\$#</code>	nombre de paramètres
<code>\$@</code> , <code>\$*</code>	tous les paramètres
<code>\$?</code>	code retour de la dernière commande
<code>\$\$</code>	PID du shell
<code>#!</code>	PID du dernier processus lancé en arrière-plan

```
echo "Ce script se nomme $0 et a reçu $# paramètre(s) : $@"
```

Afficher et saisir

```
#!/bin/bash
echo -n "Entrez votre nom : "
read nom
echo "Bonjour $nom"
printf "%s a %d ans\n" "$nom" 20
read -s -n 1 -t 5 touche      # une touche, sans écho, 5 s maximum
exit 0
```

Commandes internes utiles : `echo` , `printf` , `read` , `exit` , `let` , `eval` , `shift` , `export` , `source`

Les tests

Test	Vrai si	Test	Vrai si
<code>[-e f]</code>	f existe	<code>[-z "\$s"]</code>	chaîne vide
<code>[-f f]</code>	f est un fichier	<code>["\$a" = "\$b"]</code>	chaînes égales
<code>[-d f]</code>	f est un répertoire	<code>[\$n -eq 5]</code>	nombres égaux
<code>[-x f]</code>	f est exécutable	<code>[\$n -lt 5]</code>	n inférieur à 5

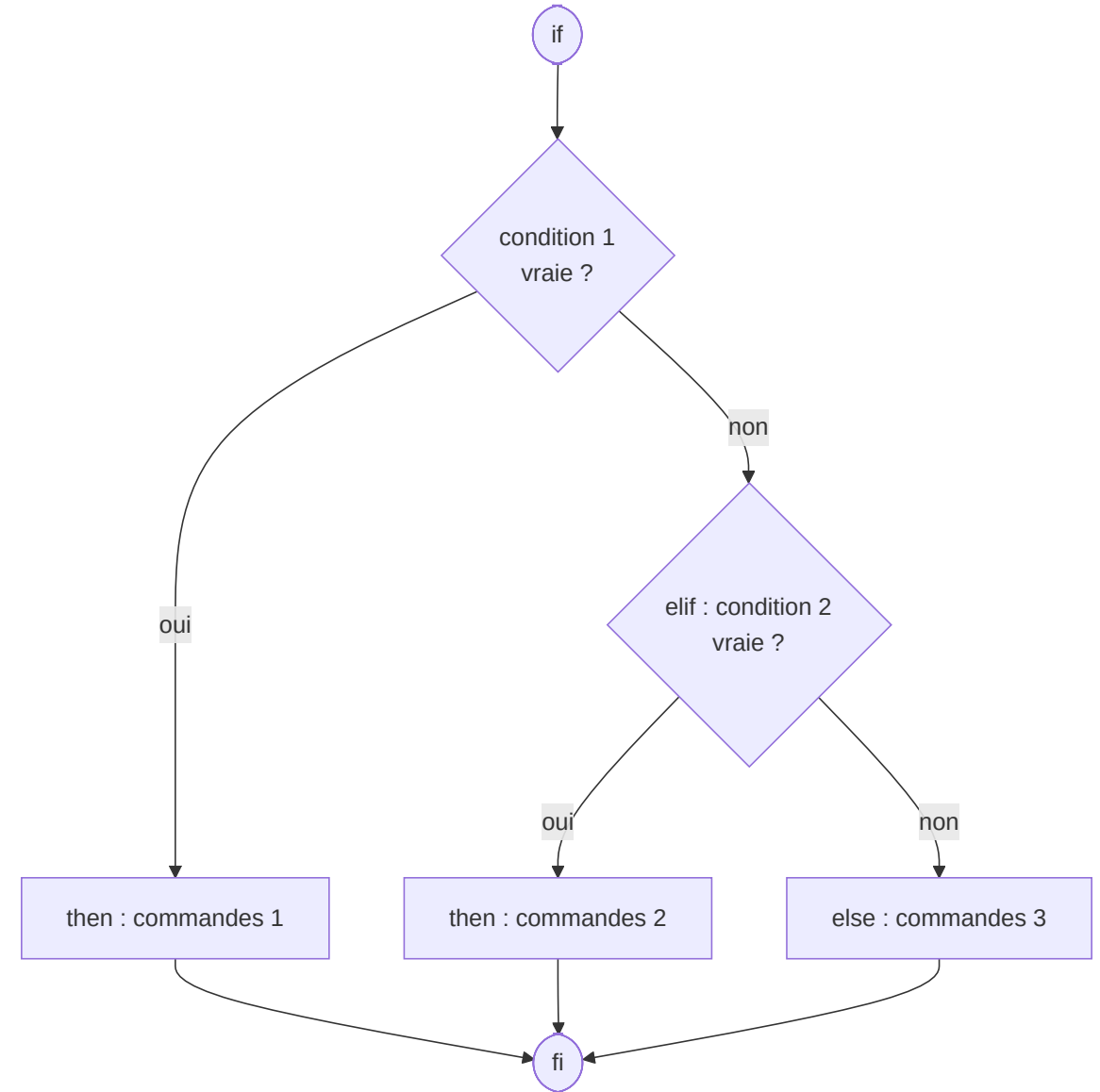
```
[[ $cp =~ ^(84|13)[0-9]{3}$ ]]      # expression régulière (ERE)
```

Des **espaces** autour de `[`, `,` `]` et des opérateurs ! Voir `help test` .

if / then / else

```
if [ ! -d "$1" ]  
then  
    mkdir "$1"  
fi  
  
if cmp a b &> /dev/null  
then echo "identiques"  
else echo "différents"  
fi
```

La condition est le code retour d'une commande : `0` = VRAI



case et select

```
case $1 in
  1|3|5|7|9) echo "chiffre impair";;
  2|4|6|8)  echo "chiffre pair";;
  0)        echo "zéro";;
  *)        echo "c'est un chiffre qu'il me faut !";;
esac

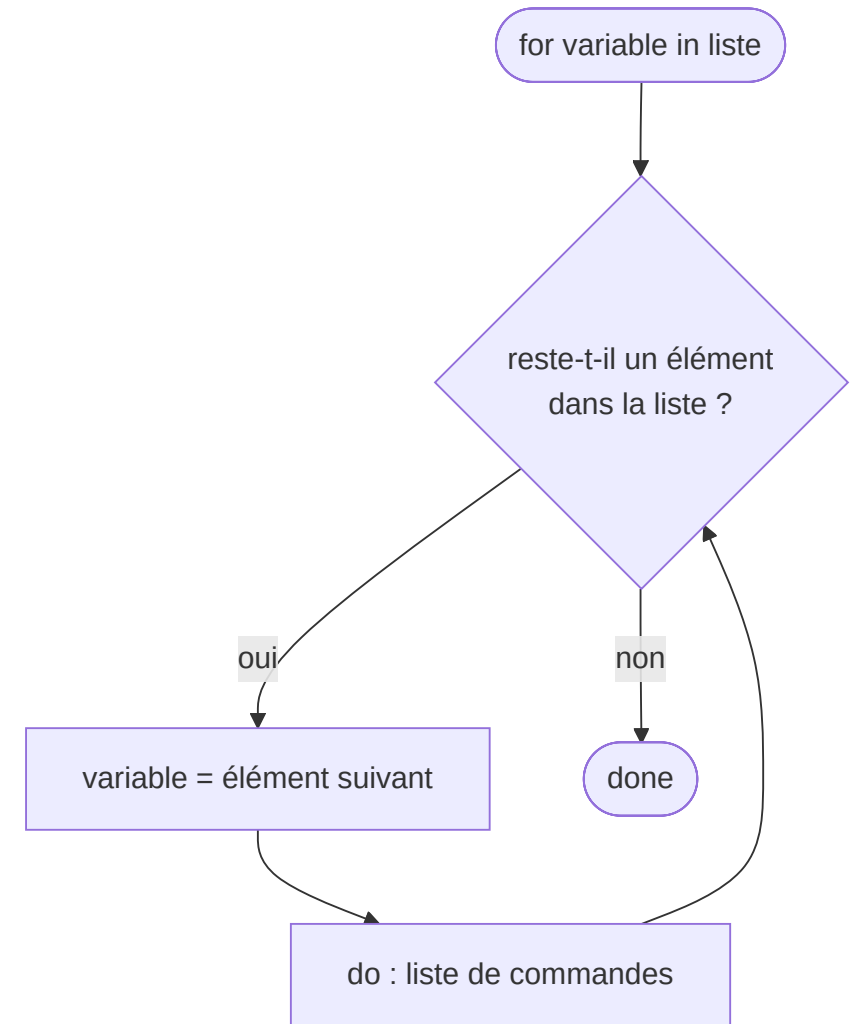
PS3='Menu : '
select choix in "création" "modification" "quitter"
do
  echo "Votre choix est $choix"
  break
done
```

La boucle for

```
for i in 1 2 3 4 5
do
    echo "i=$i"
done
```

```
for fic in *.txt
do
    echo "$fic"
done
```

```
for ((i=1; i <= 10; i++))
do
    echo -n "$i "
done
```

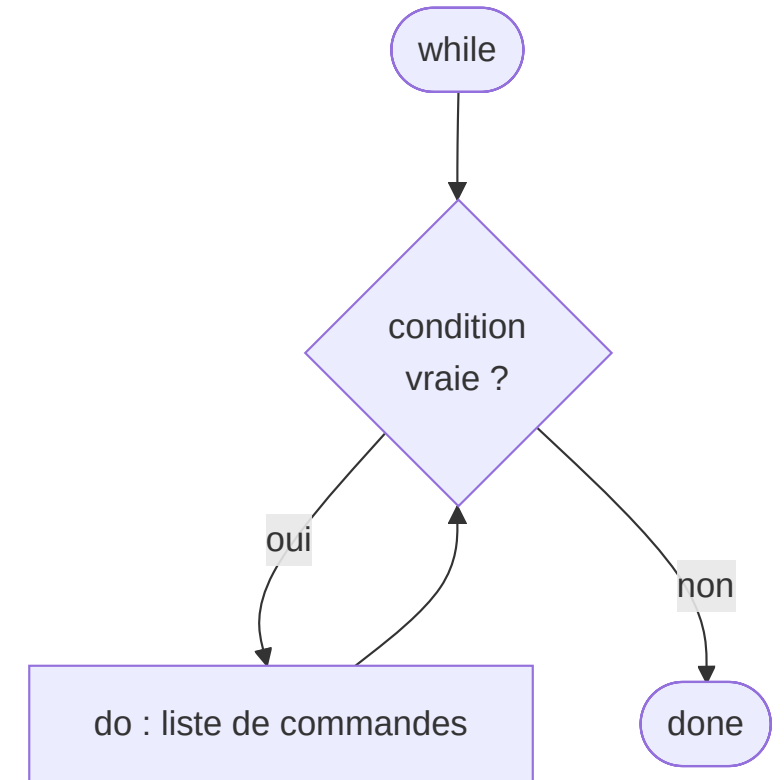


Les boucles while et until

```
a=1
while [ "$a" -le 10 ]
do
    echo -n "$a "
    ((a += 1))
done

ctr=0
until (( ctr >= 10 ))
do
    ((ctr += 1))
done
```

`until` boucle tant que la condition est **fausse** · `break` sort de la boucle · `continue` passe au tour suivant



Les fonctions

```
#!/bin/bash
root_check() {
    [ "$(id -u)" -eq 0 ]          # code retour 0 si root
}

bonjour() {
    local nom=$1                # variable locale
    echo "Bonjour $nom"
}

bonjour "$USER"
root_check || { echo "Ce script doit être lancé par root"; exit 1; }
```

Paramètres `$1` , `$2` ... · variables locales (`local`) · `return` pour le code retour

À vous de jouer !

[Commandes de base](#) · [TP « Commandes de base Linux »](#)

[TP1](#) · [TP2](#) · [TP3](#) · [TP4](#) · [TP5](#) · [Serveur LAMP](#)

Le cours complet : [README.md](#)